FACULTY OF ENGINEERING AND BASIC SCIENCES
ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE

COURSE: ETL (G01)
ETL Project - Second Delivery: Pipeline Automation, API Integration and Data Validation

1. Context

In the first delivery, you designed and implemented an analytical data pipeline using a primary dataset. You defined the grain, designed a dimensional model, built a Data Warehouse, and created KPIs and Visualizations.

In this second delivery, your goal is to **evolve that solution into a more realistic data engineering pipeline** by:

- integrating a new external data source (API)
- automating the pipeline using Airflow
- validating the transformed data using Great Expectations
- refining (if necessary) your dimensional model

2. Objective

Operationalize and extend your first delivery solution by integrating an API source, implementing a validated ETL pipeline, and orchestrating it using Airflow.

3. Data Sources

3.1. Primary Dataset (from First Delivery)

- The dataset you selected and modeled previously
- Must be reused
- Your original dimensional model must be preserved or refined

3.2. Public API (NEW SOURCE)

You must select a public API relevant to your problem.

Requirements:

- The API must provide **complementary data**
- It must add analytical value
- It must be integrated into your pipeline

You must explicitly explain how the API data enhances your analytical model.

Important: Data Integration is a Design Decision

There is no single correct way to integrate both sources.
You must define and justify:

- integration strategy
- matching keys
- handling of inconsistencies
- temporal alignment (if applicable)

4. What is Expected

4.1. Requirements Gathering (Refined)

- The purpose of this stage is to clearly define the objectives, data needs, and expected outcomes of the data analysis task before building the pipeline. This step should answer "Why are we building this pipeline?" and "What insights should the data deliver?"
- Consider the new data in the refined objectives.

4.2. Extract

- ~~Extract data from your original dataset~~
  - ~~Minimum 50,000 rows~~
  - ~~Minimum 10 attributes~~
- ~~Extract data from the API~~

4.3. EDA, Data Profiling & Quality Assessment

~~Perform a profiling of the data from the API, including:~~

- ~~Null value analysis~~
- ~~Duplicate detection~~
- ~~Data type validation~~
- ~~Outlier detection~~
- ~~Inconsistent formatting detection~~

~~Document all quality issues identified.~~

~~You must analyze how the new API data affects your existing model and integration logic.~~

4.4. Cleaning

- ~~Clean both sources~~
- ~~Handle:~~
  - ~~null values~~
  - ~~duplicates~~
  - ~~inconsistent formats~~

4.5. Transformation & Integration

- ~~Transform both datasets~~
- ~~Integrate them based on your design decisions~~

- Produce transformed data suitable for building your analytical model.

You must clearly define how records from both sources are matched and integrated.

4.6. Dimensional Modeling (MANDATORY)

You must reuse and extend your first-delivery model.

Requirements:
- Maintain or redefine the grain
- Clearly define:
  - fact table(s)
  - dimension tables
- Ensure the model supports analytical queries

You must document:
- the grain
- any changes to the model
- how the API data impacted your design

4.7. **Data Validation with Great Expectations (MANDATORY)**

You must implement validation using Great Expectations.

Requirements:
- Validation must be applied **after transformation**
- Validation must be applied to the dataset that feeds your analytical model
- Validation must be integrated as a **task in your Airflow DAG**

Control Rule:
- Critical validation failures must **stop the pipeline**
- Non-critical issues may be logged

You must design your own expectations
Your validation must reflect your design decisions, including:
- grain consistency
- uniqueness constraints
- null checks in key fields
- valid numerical ranges
- logical consistency
- etc.

Documentation required:
- list of expectations
- classification (critical vs non-critical)
- justification of each rule

4.8. Load

- Load the validated data into your Data Warehouse
- Tables must reflect your dimensional model

4.9. Orchestration (Airflow)

Your DAG must:

- represent the full pipeline
- include a validation step
- enforce dependencies
- be modular and readable

Example diagram is provided in figure 1 (overall project flow).

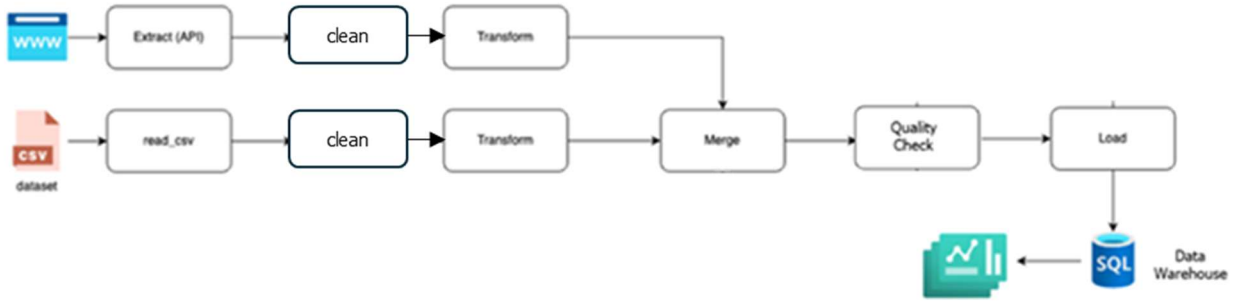

Figure 1. Block Diagram

4.10. Dashboard Requirements

- Build a dashboard connected to your Data Warehouse using tools such as: Power BI, Looker, Tableau, etc.
- The visualizations should provide business-relevant insights based on the project objectives refined.

5. Deliverables

5.1. GitHub Repository

Must include:

- Airflow DAG
- Python scripts
- Great Expectations configuration
- requirements.txt or Dockerfile

5.2. README.md (MANDATORY)

Must include:

- Refined objectives
- Summary of the data profiling of both data sources
- API details and why it was chosen,
- data integration strategy
- dimensional model design (including grain)
- validation strategy
- explanation of pipeline
- Airflow DAG design.
- Assumptions and decisions made during transformations.
- Great Expectations validation suite summary,
- Examples of visualizations and insights.

5.3. Data Warehouse

- Final tables created
- Consistent with your model

5.4. Dashboard

- Screenshots or link
- Explanation of insights

6. Technology Stack

Layer	Tool/Technology	Purpose
Orchestration	Apache Airflow	Automate and schedule ETL workflows
Data Source	Public API	Provide fresh external data
Data Storage	PostgreSQL / MySQL	Persist clean and structured data
Quality Checks	Great Expectations	Validate data before loading
Transformation	Python (Pandas)	Data cleaning and formatting
Visualization	Power BI, Locker, etc.	Insights and dashboards
Documentation & Versioning	GitHub + Markdown	Project tracking and version control

7. Evaluation

Criteria	Weight
Data Integration Strategy	1.2
Dimensional Modeling	1.2
Validation Strategy (GX)	1.2
Airflow DAG Design	1.0
Transformation Logic	0.8
Database Implementation	0.7

Criteria	Weight
Dashboard Quality	0.6
Code Quality	0.5
Documentation	0.5